

Organization X System

An Integrated Digital Management System

Project Report

Project team:

Karim Ejje (Product Owner)

Vladimer Gogia as (Scrum Master & Developer)

Harsh Dhiman (Developer)

Danna Sofía Jiménez Munetones (Developer)

Asadbek Uktamjonov (Quality Assurance)

Frontend Programming

July 2026

1. Introduction and Problem Description

Organization X is a member-based organization that, prior to this project, managed its day-to-day operations across a patchwork of disconnected tools. Member records lived in spreadsheets, organizational documents were shared ad hoc, decisions were taken informally, meetings were coordinated over chat, and finances were tracked in yet another spreadsheet. The organization also maintained an active WhatsApp group as its main communication channel. No single system tied these functions together, and the roughly four thousand legacy member records had accumulated inconsistent formatting — duplicate emails, mixed phone-number styles, and missing identifiers.

The goal of this project was to replace that fragmented setup with a single web platform that lets members serve themselves and lets administrators manage the organization from one place. The system covers six functional domains: member management, document management, electronic voting, meeting coordination, financial record-keeping, and WhatsApp communication analytics, together with an email notification facility.

The project was delivered by a five-person student team over four execution sprints, under a deliberate set of constraints: a zero-budget mandate, a server-rendered web application rather than a single-page app, and an MVP-first philosophy in which the hardest features were scoped down to honest, demonstrable minimums rather than attempted in full.

This report describes the requirements the team committed to, the architecture and database that support them, how each module was implemented, how the system was tested and evaluated, how it is deployed, and what remains as future work.

Report structure:

- Section 2 sets out the requirements and the scope boundary.
- Section 3 describes the system architecture and technology choices.
- Section 4 covers the database design.
- Section 5 details the implementation of each module.
- Section 6 presents testing and evaluation.
- Section 7 documents local and cloud deployment.
- Section 8 lists future work and known limitations.
- Section 9 provides references.

2. System Requirements

2.1 Functional requirements

The functional requirements are grouped by module. Each describes a capability the delivered system provides.

Module	Key capabilities
Members & Auth	Members register, log in, and edit their own profile. Administrators manage member records through the Django admin. Two roles exist: ADMIN and MEMBER.
Documents	Administrators upload and tag documents. Members browse, keyword-search, view, and download them. Uploads are restricted to administrators.
Voting	Administrators create ballots with options and an open/close window. Members cast exactly one vote per ballot. Results are shown only after a ballot closes.
Meetings	Administrators schedule meetings with a link and minutes. Members view upcoming and past meetings and download a calendar (.ics) file.
Finance	Administrators record income and expense transactions by category. The system shows totals and a summary chart. Finance data is admin-only.
WhatsApp analytics	Administrators upload an exported chat file; the system computes activity, peak hours, sentiment, spam flags, and top contributors, and renders a dashboard.
Notifications	Administrators send email broadcasts to members. A scheduled job sends meeting reminders automatically.
Data migration	Approximately 4,000 legacy member records are cleaned and imported via a management command.

2.2 Non-functional requirements

- Security: authenticated access, role-based gating of admin functions, and the protections Django provides by default (CSRF, ORM-level SQL-injection resistance, password hashing).
- Usability & delivery: responsive server-rendered web pages (no native mobile app); a single interface language.
- Data protection: all data used in development and demonstrations is synthetic, no real personal information is stored.
- Maintainability: one repository, one Django project, conventional migrations, and code review on every change.

2.3 Scope boundary

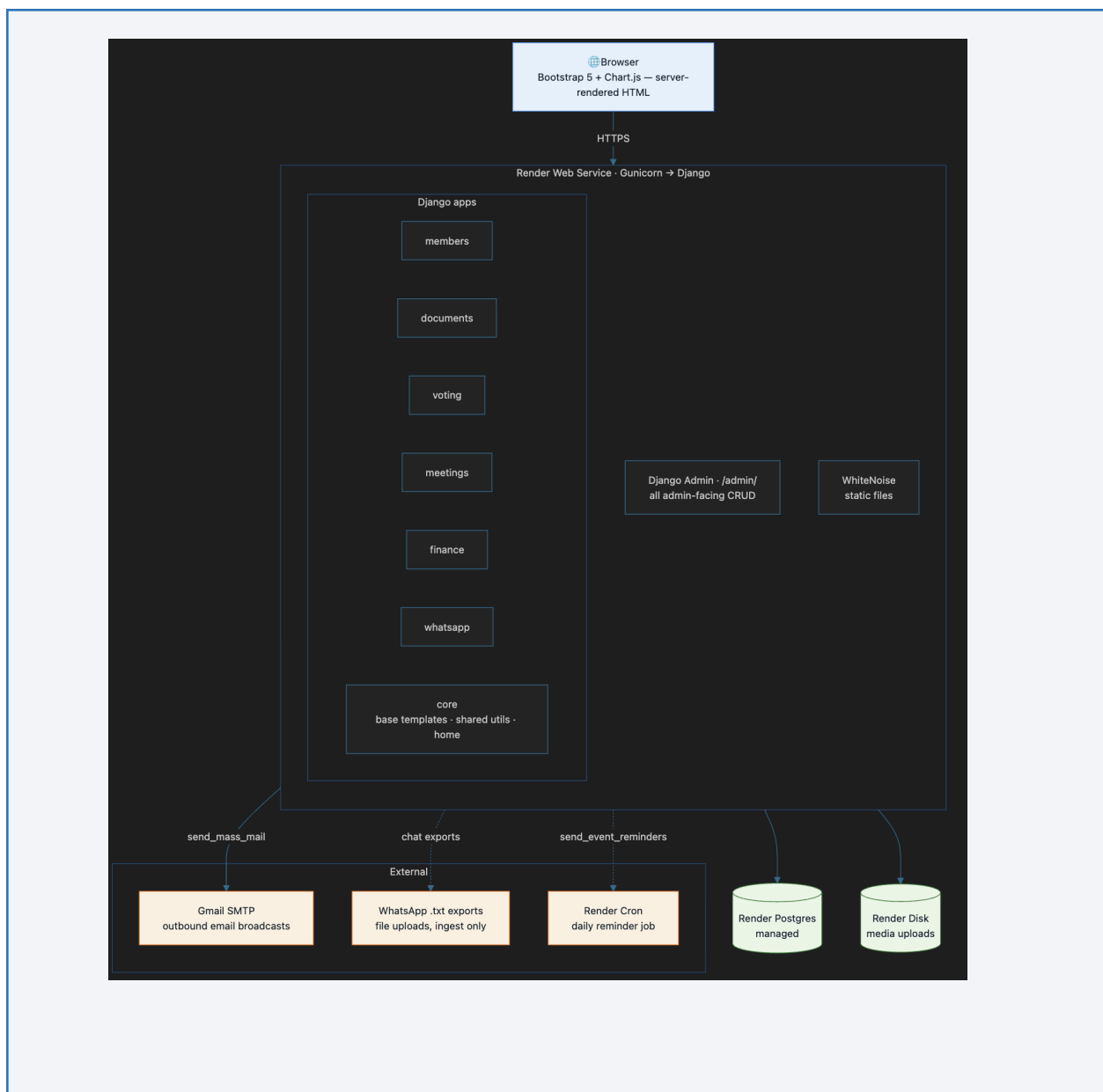
A defining decision of the project was to scope the hardest items down to demonstrable minimums. The table below summarizes what was built versus what was deliberately cut or simplified; Section 8 revisits the cut items as future work.

In scope (v1)	Cut or simplified
1. Member self-service + admin management 2. Document upload, tag, search, download 3. E-voting with one-vote enforcement 4. Meeting list + minutes + .ics 5. Finance income/expense + summary 6. WhatsApp analytics from exported chats	1. Live WhatsApp Business API integration 2. Intelligent automated replies 3. Two-factor auth; email password reset 4. Video/meeting hosting (simplified to a link field) 5. BI-style financial reporting (simplified) 6. Real-time features; document versioning (cut)

3. System Architecture

3.1 Overview

The system is a monolithic, server-rendered Django application: a single repository, a single deployable process, and one PostgreSQL database (SQLite in local development). A browser request is routed by Django's URL dispatcher to a view function, which queries the ORM and renders a Bootstrap-styled HTML template. Administrative create/read/update/delete for most models is handled by the built-in Django admin rather than custom pages, which keeps the surface area small. Client-side interactivity is limited to Chart.js visualizations and a basic JavaScript.



3.2 Application modules

The project is organized into focused Django apps, each owning its models, views, templates, and admin registration:

Django app	Responsibility
members	Custom user model with roles; registration, login, profile; member import.
documents	Document and Tag models; upload, tagging, full-text search, download.
voting	Ballot, options, and votes; casting; results and tallies.
meetings	Meeting scheduling, detail, minutes, and .ics calendar export.
finance	Categories and transactions; summary totals and chart.
whatsapp	Chat export upload and offline analytics (sentiment, peaks, spam, influencers).
notifications	Email broadcasts and the scheduled meeting-reminder command.
core	Shared base templates, error pages, decorators, and management commands (seed/backup).
config	Project settings, root URL configuration, and WSGI entry point.

3.3 Technology stack and rationale

Django + PostgreSQL + Bootstrap 5 form the core. Django was chosen over a lighter framework such as Flask specifically because it supplies exactly the components an inexperienced team would otherwise reimplement poorly.

The stack is completed by **Chart.js** for charts, **pandas** and **vaderSentiment** for offline analytics, **unicorn** as the production WSGI server and **django-crispy-forms** for Bootstrap

form rendering. All dependencies are pinned in requirements.txt.

3.4 Authentication and authorization

Authentication uses Django's session framework over a custom user model (members.User extends AbstractUser) that adds a role field, a member identifier, and profile fields. Authorization is enforced with two decorators: the standard `@login_required` for member-facing pages, and a project-specific `@admin_required` (in `core/decorators.py`) for administrative views such as document upload and the finance pages. A signal keeps an "Org Admins" Django group in sync with each user's role, so admin-group permissions follow the role automatically.

3.5 Key request flows

- Documents: an admin uploads a tagged file; a member searches by keyword (PostgreSQL full-text search), opens the detail page, and downloads the file.
- Voting (integrity-critical): a member opens an open ballot, casts one vote; the view re-checks the open/close window and one-vote constraint at submission time before creating the Vote row. Results are withheld until the ballot closes.
- WhatsApp analytics: an admin uploads an exported .txt chat; the analytics module parses it into a table and computes sentiment, peak hours, spam flags, and top influencers, persisting the result as JSON for the dashboard to render.
- Notifications: an admin composes an email broadcast to a member segment; a separate daily cron command sends reminders for meetings scheduled in the next window.

4. Database Design

4.1 Entity-relationship overview

The schema is normalized around the custom user model, which is referenced by nearly every other table as the actor who created or owns a record. Each functional app contributes its own small cluster of tables. The detailed ERD-diagram in the github repository shows the details on the entities and their relationships.

4.2 Core tables

The principal models and their notable fields, as implemented:

Model	Notable fields	Notes
members.User	role, member_id, phone, joined_at	Extends AbstractUser; roles ADMIN/MEMBER.
documents.Document	title, file, tags (M2M), uploaded_by, uploaded_at	Tag is a separate model with a slug.
voting.Ballot	title, opens_at, closes_at, created_by, is_active	Defines the voting window.
voting.Vote	ballot, option, voter, cast_at	unique_together (ballot, voter); FKs use PROTECT.
meetings.Meeting	scheduled_at, duration_minutes, location_or_link, minutes, status	Link field replaces video hosting.
finance.Transaction	amount (Decimal), transaction_date, type, category, recorded_by	Category on_delete = SET_NULL.
whatsapp.Chat Analysis	chat_export, generated_at, results (JSON)	Analytics output stored as JSON.
notifications.Broadcast	recipient_filter, recipient_count	Tracks sent email broadcasts.

4.3 Integrity and delete policies

The database enforces the one-vote-per-member rule directly through a unique constraint on (ballot, voter) in the Vote table, so a race between two submissions cannot produce two vote rows — the second insert fails at the database layer and is handled gracefully by the view. Delete behavior is chosen per relationship: votes and ballot options use PROTECT so historical results cannot be silently destroyed; a transaction's category uses SET_NULL so removing a category preserves the financial record; and a chat analysis cascades from its parent export. These choices favor preserving audit-relevant data over convenience.

4.4 Migrations and seed data

All schema changes ship as Django migrations committed to the repository, keeping every developer's database in step. Two management commands support the demo: `import_members` cleans and loads the ~4,000-row legacy export, and `seed_data` recreates a small, consistent demo dataset (administrators, members, sample ballots, documents, meetings, and finance entries) in under a minute so a demonstration can be restored with one command.

5. Implementation Details

This section summarizes how each module was built and who owned it. The application follows a consistent pattern per app: models, admin registration, views with the appropriate access decorator, and Bootstrap templates. So the descriptions focus on what is distinctive.

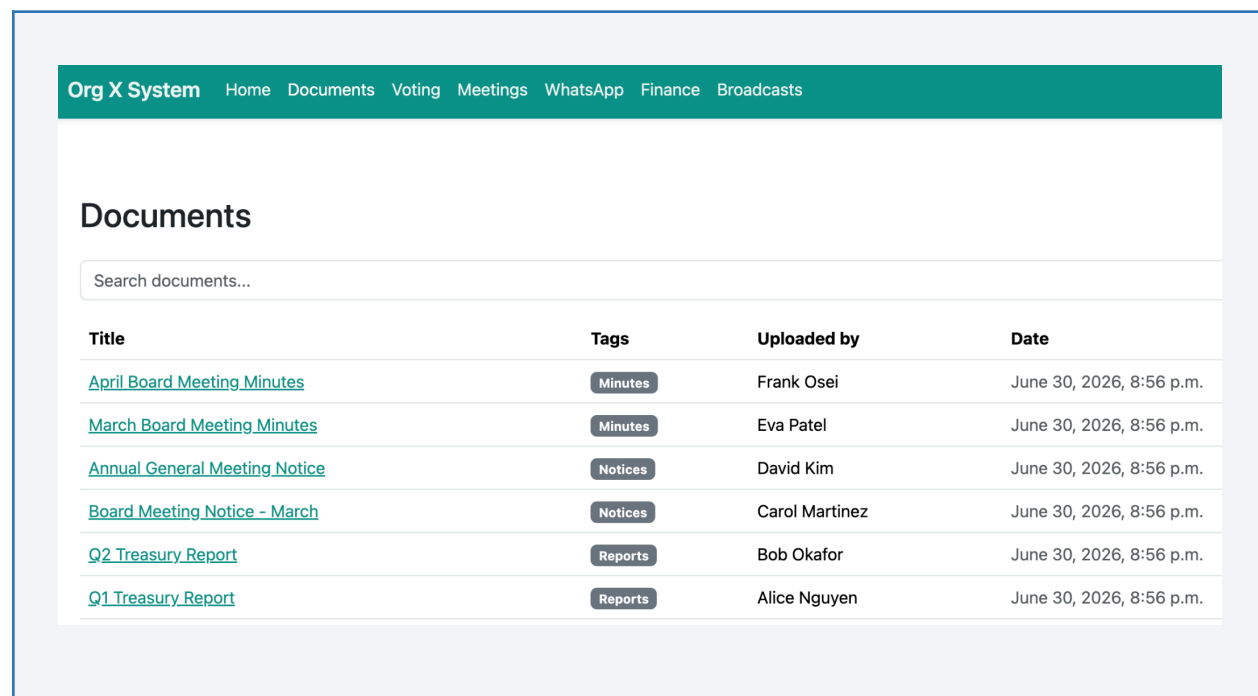
5.1 Members, authentication, and data migration

(Harsh) The members app defines the custom user model and wires Django's authentication for registration, login, logout, and profile editing. Administrators manage members through a configured admin list.

The 4,000-record migration is an `import_members` management command that cleans the legacy export — de-duplicating emails, standardizing phone formats, and normalizing identifiers — and loads it idempotently so re-running does not create duplicates.

5.2 Documents

(Harsh) The documents app stores files with a many-to-many tag relationship. Members search by keyword using PostgreSQL full-text search over the title and metadata, open a detail page, and download. Upload is gated to administrators by the `@admin_required` decorator, with the upload control hidden from non-admins in the template.

The screenshot shows a web application interface for 'Org X System'. At the top is a teal navigation bar with links: Home, Documents, Voting, Meetings, WhatsApp, Finance, and Broadcasts. Below the navigation bar is a section titled 'Documents'. Under this title is a search bar with the placeholder text 'Search documents...'. Below the search bar is a table with four columns: 'Title', 'Tags', 'Uploaded by', and 'Date'. The table contains six rows of document entries. Each row has a link for the title, a tag button, the name of the uploader, and the upload date and time. The tags are 'Minutes', 'Notices', and 'Reports'.

Title	Tags	Uploaded by	Date
April Board Meeting Minutes	Minutes	Frank Osei	June 30, 2026, 8:56 p.m.
March Board Meeting Minutes	Minutes	Eva Patel	June 30, 2026, 8:56 p.m.
Annual General Meeting Notice	Notices	David Kim	June 30, 2026, 8:56 p.m.
Board Meeting Notice - March	Notices	Carol Martinez	June 30, 2026, 8:56 p.m.
Q2 Treasury Report	Reports	Bob Okafor	June 30, 2026, 8:56 p.m.
Q1 Treasury Report	Reports	Alice Nguyen	June 30, 2026, 8:56 p.m.

5.3 Voting

(Vladimer) The voting app models a Ballot with ordered options and Vote rows constrained to one per member. The ballot list separates open from closed ballots and marks those the

member has already voted on. Casting is a POST-only view that re-validates the voting window and the one-vote rule at submission, then creates the Vote inside a transaction so a duplicate submission yields a friendly "already voted" message rather than an error. Results, including a Chart.js bar chart, appear only after the ballot closes.

Who will win the world cup

Open until July 18, 2026, 10:21 a.m.

☐ Argentina

☒ France

☐ Columbia

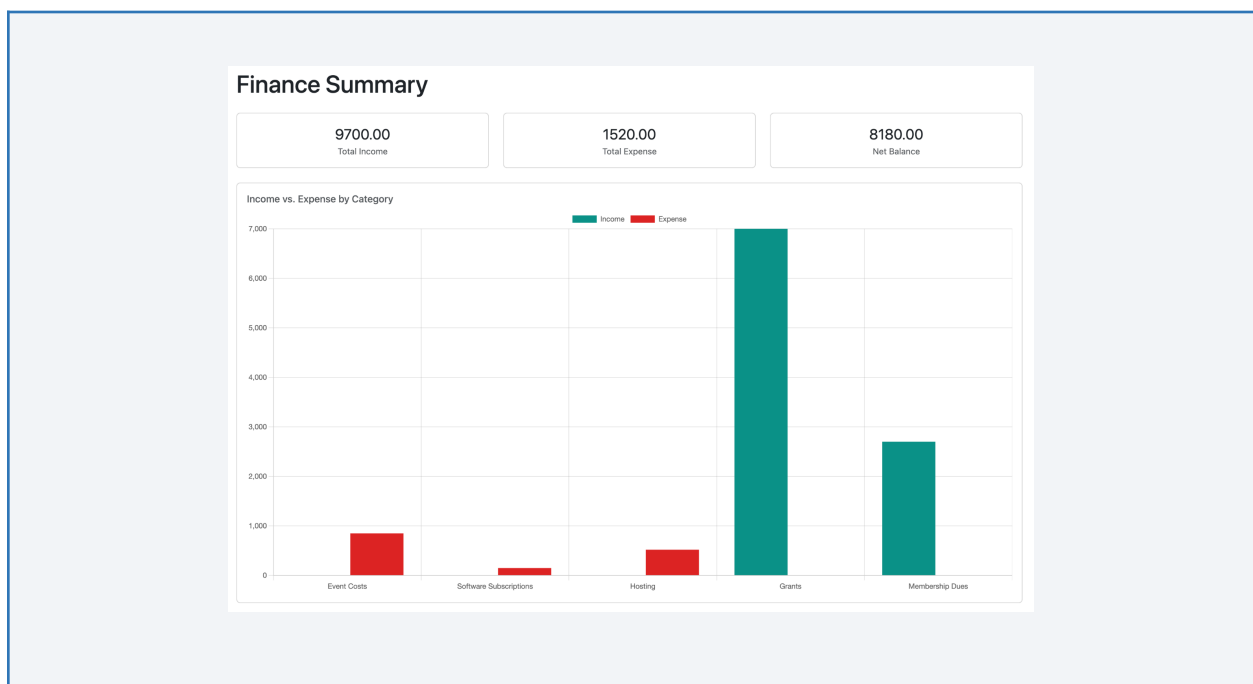
☐ Cabo Verde

Submit voteCancel

Your vote is recorded against your account and is not anonymous.

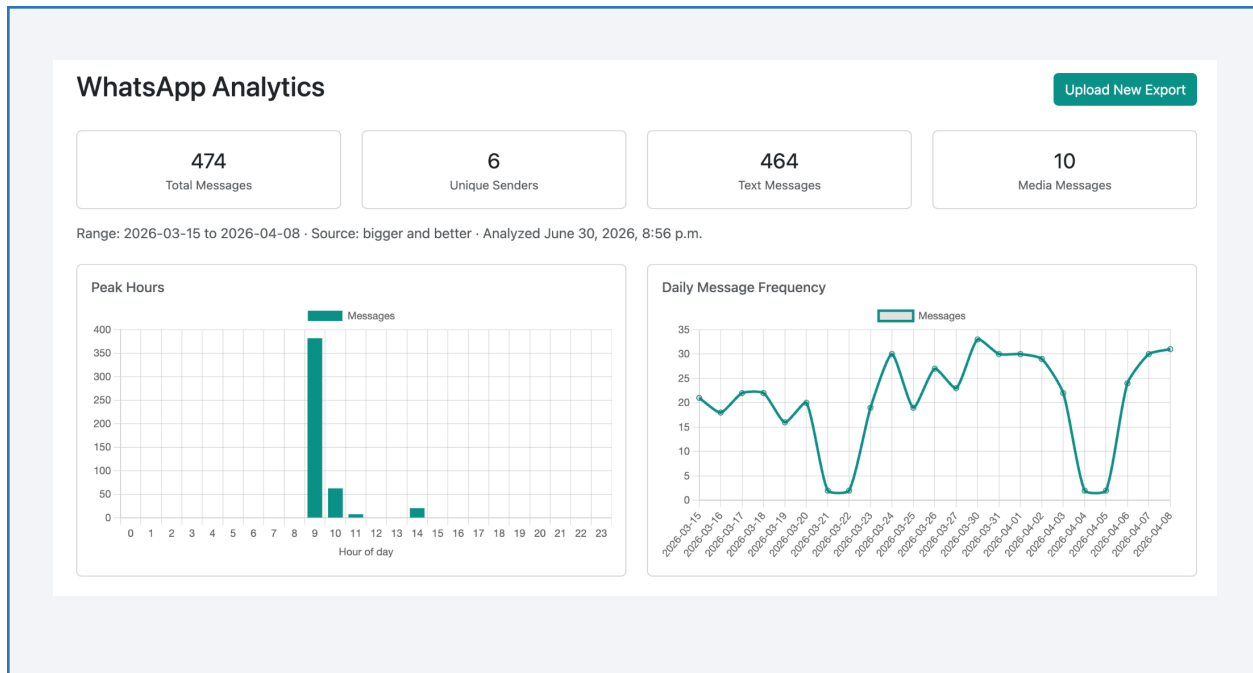
5.4 Meetings, finance, and shared UI

(Danna) The meetings app lists upcoming and past meetings, shows a detail page with minutes, and generates a downloadable .ics calendar file. The finance app records income and expense transactions by category and presents monthly totals with a Chart.js summary chart. Danna also owned the shared UI — the base template, navigation bar, Bootstrap theme, home dashboard, and the 404/500 error pages — which keeps the modules visually consistent.



5.5 WhatsApp analytics

(Vladimer) The whatsapp app accepts an exported chat file, parses it into a pandas table, and computes sentiment (via VADER), peak activity hours, a keyword/frequency spam heuristic, and the top contributors by message and reply volume. Results are persisted as JSON on a ChatAnalysis record and rendered as a dashboard of Chart.js visualizations. The analysis runs offline on the uploaded export rather than integrating with the live WhatsApp API.



5.6 Notifications

(Harsh) The notifications app provides an admin email-broadcast composer targeting a member segment, and a `send_meeting_reminders` management command scheduled as a daily cron job on the host to remind members of upcoming meetings. Sent broadcasts are recorded with their recipient count.

5.7 Demo data

(Karim) supports all of the above: the `seed_data` command produces a coherent set of administrators, members, and sample content so any flow can be demonstrated immediately.

6. Testing and Evaluation

6.1 Approach

Quality was split across two roles. The Product Owner wrote test scenarios in a Given/When/Then style directly from the acceptance criteria; the QA engineer, Asadbek, turned those scenarios into concrete, click-by-click test cases, executed them on each release, reproduced and filed bugs on a Jira board (Open → In Progress → Fixed → Verified → Deferred), and verified fixes. This division let scenarios be authored by whoever held the requirements and executed by whoever owned the release.

6.2 Automated tests

The repository includes Django test modules across the apps (members, documents, voting, meetings, finance, whatsapp, core). Coverage is targeted rather than exhaustive — it focuses on the highest-risk behavior such as the voting constraints and the analytics output shape. The team treats broadening automated coverage, especially access-control regression tests, as an explicit follow-up.

6.3 Manual test cases

Manual cases exercised each end-to-end flow: register → log in → edit profile; upload → search → download a document; cast a vote and confirm the one-vote rule; schedule and download a meeting; record a transaction and see totals update; and upload a chat export and view the analytics dashboard. Each case recorded expected versus actual results, and failures became Jira bugs triaged by priority.

6.4 Security evaluation

Security relied first on Django's defaults — CSRF protection on forms, ORM-parameterized queries that resist SQL injection, and hashed passwords — verified as enabled during a dedicated security pass. That pass also checked production configuration: DEBUG disabled, ALLOWED_HOSTS restricted, the secret key supplied from the environment, and the broadcast composer tested for cross-site scripting.

Code review caught real issues before release, which is itself evidence the process worked. Two recurring classes are worth recording honestly:

1. administrative access gates on the finance and document views were unintentionally weakened in more than one pull request and had to be restored,
2. an email TLS-verification relaxation intended only for local development risked leaking into production configuration.

Both were identified in review and corrected, and the team added the recommendation to encode the admin-gate expectations as automated tests so any regression fails the build.

6.5 Results and known issues

By the finalization sprint the core flows passed their manual test cases and the ballot integrity behavior held under duplicate-submission testing. Remaining items were consciously deferred rather than treated as blockers: broader automated coverage, and the production-readiness gaps noted in Section 8. The QA sign-off for the demo was go, with a documented list of known limitations.

7. Deployment

The application runs in two environments: each developer's machine for day-to-day work, and Render as a hosted URL for stakeholders. The final live demonstration is run from a local machine tunneled to a public URL to avoid free-tier cold starts.

7.1 Local deployment

The runbook brings the app up on any machine in a few minutes. The essential sequence is:

- Clone the repository and create a Python virtual environment.
- Install dependencies: `pip install -r requirements.txt`.
- Create a `.env` file (a `.env.example` is provided); locally this can point at SQLite, so no database server is required.
- Apply the schema: `python manage.py migrate`.
- Create an administrator: `python manage.py createsuperuser`.
- Load demo content: `python manage.py seed_data`.
- Run the server: `python manage.py runserver`, then open the local URL.

7.2 Render (cloud) deployment

Deployment to Render is described declaratively in `render.yaml`. A web service builds with `pip install`, `collectstatic`, and `migrate`, and starts under `gunicorn` (`config.wsgi`). Configuration is supplied through environment variables: an auto-generated `SECRET_KEY`, `DEBUG` set to `False`, `ALLOWED_HOSTS` limited to the Render domain, a `DATABASE_URL` injected from a managed free-tier PostgreSQL instance, and the email host and credentials. `WhiteNoise` serves static files. A separate daily cron service runs the `send_meeting_reminders` command. Because the free web service sleeps after inactivity and incurs a cold start, the team hits the URL shortly before any stakeholder meeting and runs the actual final demo locally over a tunnel for reliability.

8. Future Work and Known Limitations

8.1 Deferred features (roadmap)

Several capabilities from the original brief were deliberately cut from the MVP and remain candidates for future iterations:

- Live WhatsApp Business API integration, replacing the current exported-file workflow.
- Intelligent automated replies to WhatsApp messages.
- Two-factor authentication and self-service password reset by email.
- Document version history and granular per-resource permissions.
- Real-time features (live vote counts, live chat) and a multilingual interface.

8.2 Production-readiness gaps

The following are understood limitations of the free-tier, demo-oriented deployment rather than defects:

- Email volume: a personal Gmail account allows on the order of 500 messages per day, which does not cover a broadcast to the full 4,000-member set; a transactional email provider would be needed for production.
- Synchronous broadcast sending should be moved to a background job or queue so large sends do not block a web request.
- Free-tier hosting introduces cold starts and limited database retention; a paid tier or external backup would be required for a system of record.

8.3 Engineering follow-ups

- Add access-control regression tests (non-admins receive 403 on finance and document-upload endpoints) so the recurring admin-gate issue cannot resurface silently.
- Broaden automated test coverage around the analytics and notification paths.

9. References

Framework, library and project documentation:

- Django (web framework and admin): <https://docs.djangoproject.com/>
- PostgreSQL (database and full-text search): <https://www.postgresql.org/docs/>
- Bootstrap 5 (UI framework): <https://getbootstrap.com/docs/>
- Chart.js (charts): <https://www.chartjs.org/docs/>
- VADER sentiment analysis: <https://github.com/cjhutto/vaderSentiment>
- pandas (data processing): <https://pandas.pydata.org/docs/>
- unicorn, WhiteNoise, django-crispy-forms, django-environ (project dependencies).
- Internal project artifacts (available on github): Project Plan, Architecture document, Database ERD, developer runbook, and the sprint boards maintained in the team's Jira workspace.